

Demo Video Script (English)

Track 2: Development & Local Deployment of Private AI Agents

Target runtime: 3–4 minutes | Resolution: 1920×1080 | Frame rate: 60 fps

Hook rule: project name and the "100% offline" selling point must land within the first 3 seconds.

Production notes that apply to every shot

- Record with OBS Studio or Bandicam at 1080p 60 fps; capture the browser window plus a clean capture of `rocm-smi` where required.
 - UI text is dark "precision instrument" theme with AMD red accents. Do not add flashy transitions, glows, or overlays — cut cleanly or use a 0.3 s fade in/out.
 - Subtitles/captions below screen center in the format shown; keep the same wording as the narration for readability.
 - Background music: low-volume minimal electronic or ambient pad. It must never compete with the narration.
-

Shot 1 • Opening (0:00–0:15)

Visual

Dark desktop. A browser opens `http://localhost:7860` and settles on the empty-state screen of Private RAG Agent. Behind the empty state, a thin animated RAG pipeline diagram: `QUERY → VECTOR DB → GROUNDED → ANSWER`. Fade to title card: Private RAG Agent.

On-screen caption

100% LOCAL INFERENCE • WORKS OFFLINE • DATA NEVER LEAVES THIS MACHINE

Narration

This is Private RAG Agent — a fully offline, local, private AI agent. Every document you import is processed right on this machine. Nothing ever leaves your device.

Production note

- First three seconds must show the project name and the offline claim; do not spend time on window-opening niceties.
 - Pre-record the pipeline animation and composite it behind the empty state, or animate it with CSS before the take.
-

Shot 2 • Document Ingestion (0:15–0:45)

Visual

Sidebar action "Import documents". The operator selects the six technical documents (architecture, technical whitepaper, AMD optimization and performance, deployment guide, sample project, and competition plan). One click. A progress sweep runs. The knowledge-base badge in the header updates to 6 documents · 15 chunks.

On-screen annotation

During the chunking sweep, flash the parameters `chunk 512 + overlap 64` beside the animation.

Narration

Private RAG Agent ingests PDF, Word, Markdown, Excel, and PowerPoint. Import does everything in one pass: parsing, intelligent chunking into overlapping segments, and vector embedding into a local vector store. Six documents, one click — and they are now part of a knowledge base that exists only on this machine.

Production note

- Let the on-screen chunk-count update be a visible, deliberate moment — this is the user's first concrete evidence that the pipeline ran.
 - If recording against the pre-loaded knowledge base, reset the store first so the badge visibly changes from 0 to 6 documents / 15 chunks.
-

Shot 3 • Single-Agent Q&A (0:45–1:30)

Visual

The operator types: `What technology stack does my project use?` and presses Enter. Three beats, in sequence:

- Live tool-trace panel on the right — a `rag_search` tool-call event pops in, one line at a time.
- The answer streams out in typewriter style. When it finishes, citation source cards appear under it: file name plus a relevance bar.
- The operator clicks a citation. A source-drawer slides in from the right showing the original document text, with the matched query terms highlighted in red in place.

On-screen caption

HYBRID RETRIEVAL: semantic vectors + BM25 keywords → RRF fusion → cross-encoder rerank

Narration

Before it answers anything, the agent retrieves from the knowledge base. It is a hybrid pipeline — semantic vector search with `bge-m3` embeddings, fused with BM25 keyword recall, then re-ranked by a cross-encoder model so the most relevant passages surface first. Every answer carries clickable citations. Open one, and you see the exact source text with the matched keywords highlighted in place.

Production note

- Keep the three beats readable: let the tool trace finish, let the answer finish, then trigger the drawer. Do not rush the mouse to the citation before the relevance bars render.
 - Trim the idle waiting between the Enter keypress and the first tool-trace line in the edit.
-

Shot 4 • Multi-Agent Parallel (1:30–2:30)

Visual

The operator switches to Multi-Agent Parallel mode and enters: `Analyze this project's technology stack, optimization points, and deployment method.` This is the climax — hold a slow, steady pace.

- A plan panel appears at the top: three independent subtasks with a progress bar advancing $\frac{1}{3}$ → $\frac{2}{3}$ → $\frac{3}{3}$.
- In the activity panel on the right, three researcher agents stream their retrieval events concurrently — three interleaved `rag_search` traces from Researcher 1, 2, and 3.

- As each subtask completes, a fact-check card pops in: `grounding 0.68` • SUPPORTED. Unsupported statements are honestly flagged rather than hidden.
- A short synthesizer phase, then the final assembled answer appears.

On-screen caption

DECOMPOSE → PARALLEL RESEARCH → FACT-CHECK → SYNTHESIZE • only verified content is kept

Narration

For a complex question, the system decomposes it into independent subtasks and researches them in parallel. Look at the right — three researcher agents working at once, each retrieving and verifying its own slice. This is the core design of the project. Every subtask is then passed through a fact-checker, which scores whether each statement is actually grounded in the retrieved source text; anything unsupported is honestly marked. Only content that passes verification is ever synthesized into the final answer.

Production note

- This shot carries the project thesis ("rigor" narrative). Interleaving the three tool traces on screen is essential — cut to a tight view of the activity panel if the full-screen layout reads too small.
- If the grounding score varies per subtask, that is good: it makes the verification look real rather than scripted. Do not re-take to force a perfect 1.0.

Shot 5 • GPU Acceleration (2:30–3:00)

Visual

The right panel switches to the GPU Monitor tab. Real-time curves for utilization, VRAM, and temperature pulse as inference runs. Overlay a terminal split-screen: `rocm-smi` output and the tail of `python benchmarks/bench_amd.py` printing tokens/s.

On-screen annotation

A small comparison table: CPU ≈ 1.8 tokens/s → AMD Radeon GPU + ROCm ×5–10 throughput.

Narration

All inference runs on AMD Radeon hardware. Switch to the GPU monitor and you see utilization, memory, and temperature tracking the workload in real time. On Radeon with ROCm, the same model is projected to run five to ten times faster than CPU — the measured numbers are in the benchmark section of this submission. That is exactly why the multi-agent pipeline matters — several inferences are queued in parallel and scheduled onto the GPU at once, which collapses the wall-clock time of multi-step research. For precision, Q8_0 at about eight gigabytes of VRAM; for speed, the Q4_K_M quant at about four-point-six — which is our recommended default.

Production note

- On a Radeon Cloud instance, capture this section live on the cloud GPU with `rocm-smi` and `bench_amd.py` running in the terminal.
- If a Radeon GPU is unavailable during recording, see the fallback plan in Recording Notes below — never fake the numbers.

Shot 6 • Closing (3:00–3:20)

Visual

Return to the answer view of the previous multi-agent response. Slowly scroll down to the citation cards, open the source drawer, and freeze on that frame. Fade to black with the project name and tagline centered.

On-screen caption

RETRIEVAL IS TRACEABLE • ANSWERS ARE VERIFIABLE • DATA STAYS ON THIS MACHINE

Narration

Private RAG Agent: retrieval you can trace, answers you can verify, and data that never leaves your machine. Full source code, architecture documentation, and the AMD ROCm benchmark report are submitted alongside this video.

Production note

- The freeze frame is the final image of the video; hold it at least two seconds before the title card fades in.

Recording Notes

Tooling

- Record with OBS Studio or Bandicam: 1920×1080, 60 fps, high bitrate (16–24 Mbps), MP4/H.264. Record system audio separately from any voice mic for clean mixing.
- Narrate after recording if easier: read the script lines to a click-track generated from the shot timings, or voice-over against the rendered cut.

If the Radeon Cloud GPU is unavailable at recording time

- Record Shots 1–4 fully live against the local CPU deployment — the functional pipeline (ingestion, single-agent, multi-agent with fact-checking) runs without a GPU.
- For Shot 5, do not leave a hole and do not invent numbers. Instead assemble it from real artifacts:
- A screenshot of `rocm-smi` output from the Radeon Cloud session (GPU name, utilization, VRAM, temperature).
- The tokens/s report printed by `python benchmarks/bench_amd.py` on the cloud instance.
- A screenshot of the Radeon Cloud console / GPU dashboard.
- The in-app GPU Monitor falls back to demo data when no AMD GPU is detected (the panel labels it "demo"). If you must show the panel in the cut, keep the demo-label visible and pair it with the real `rocm-smi` evidence so the data is never misrepresented.

Cutting checklist

- Fades in/out: 0.3 s; no bling transitions; keep the "precision instrument" tone.
- Confirm every caption matches its narration; captions shown on screen are the ones listed per shot.
- Runtime target 3:20–3:40. If over, trim Shot 3 idle time first, then Shot 2's progress sweep.